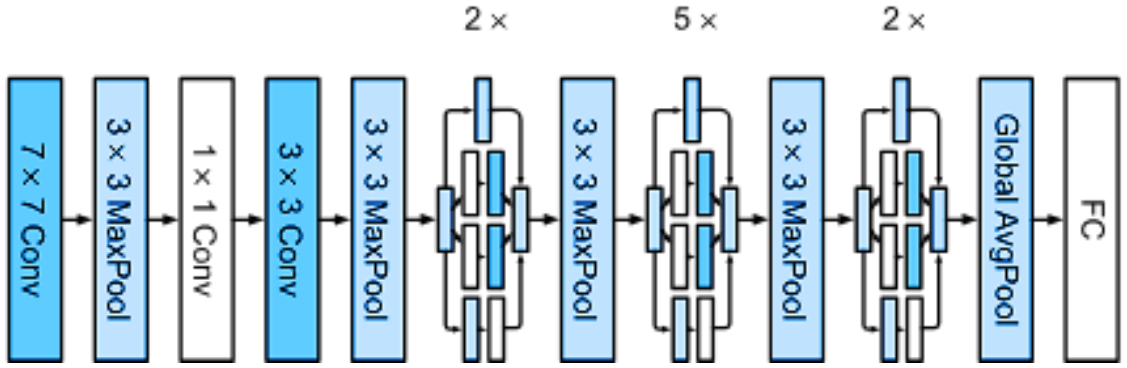
Deep Learning (23AID211)

InceptionNet (GoogleNet)
MobileNet
EfficientNet

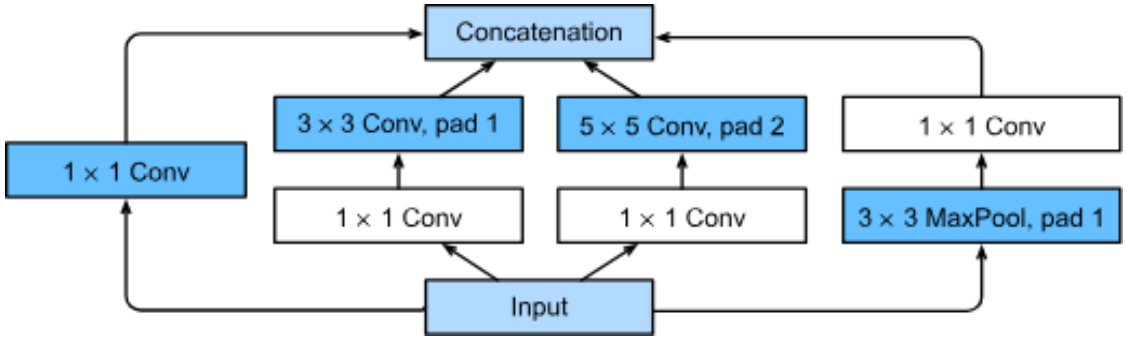
By
Dr. Vivek Patel
(B.Tech, M.Tech, Ph.D)
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham
Faridabad Campus, Delhi NCR, India

GoogLeNet (InceptionNet)-Winner of ILSVRC 2014 in both classification and detection tasks

- In 2014, GoogLeNet won the ImageNet Challenge (Szegedy et al., 2015), adapting the strengths of NiN (Lin et al., 2013), repeated blocks (Simonyan et al., 2014), with a cocktail of convolution kernels.
- It was arguably also the first network to exhibit a clear distinction among the stem (data ingestion), body (data processing), and head (prediction) components of a CNN.

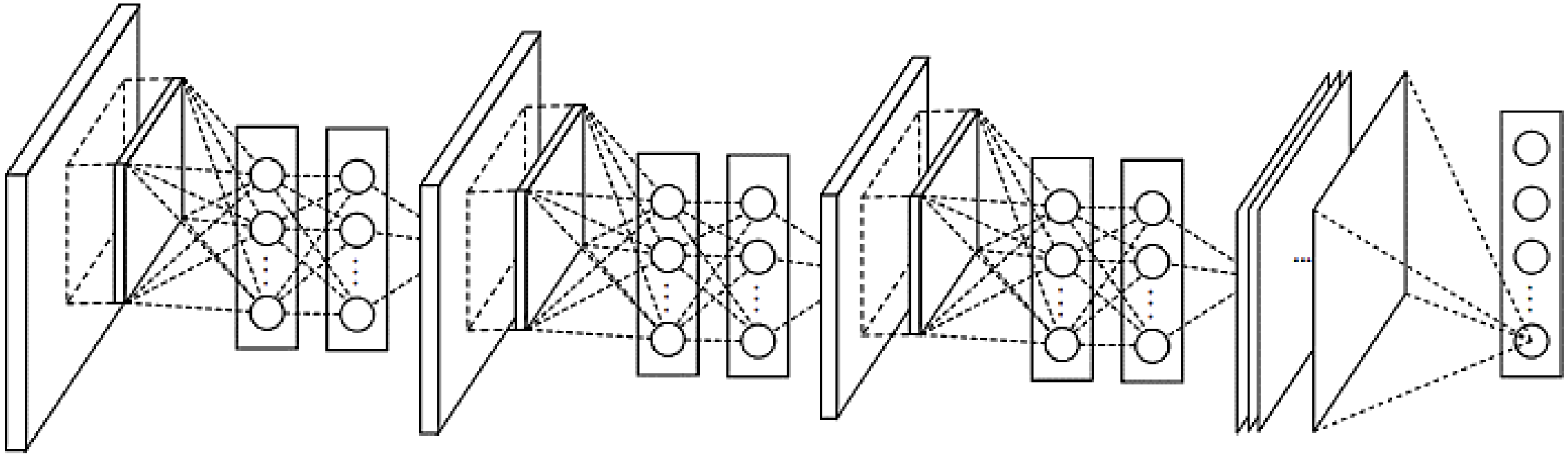


The GoogLeNet architecture



Structure of the Inception block

Network in Network (NiN)



Network in Network Architecture: https://arxiv.org/abs/1312.4400

❖ The overall structure of the NIN is the stacking of multiple mlpconv layers. It is called “Network In Network” (NIN) as we have micro networks (MLP), which are composing elements of the overall deep network, within mlpconv layers.

Key highlights of the architecture:

- **Input Layer**: Accepts a 224×224 RGB image as input.
- **Initial Convolutions and Pooling**: Applies a series of standard convolutional and max pooling layers to downsample the input and extract low-level features.
- **Local Response Normalization (LRN)**: Normalizes the feature maps early in the network to improve generalization.
- **Inception Modules**: Each module processes the input through 1×1, 3×3, and 5×5 convolutions, as well as 3×3 max pooling, all in parallel. The outputs are concatenated along the depth dimension, allowing the network to capture both fine and coarse features.

Conti…

- **Auxiliary Classifiers**: Appear as smaller branches connected to intermediate layers of the network. Include average pooling, 1×1 convolutions, fully connected layers, and softmax outputs.
- **Final Layers**: Uses global average pooling (7×7) to reduce each feature map to a single value. Followed by a fully connected layer and a softmax activation to produce the final classification output.

GoogLeNet is a **22-layer deep network** (excluding pooling layers) that emphasizes computational efficiency, making it feasible to run even on hardware with limited resources. Below is Layer by Layer architectural details of GoogLeNet.

Conti…

Inception Module

The Inception module is the architectural core of GoogLeNet. It processes the input using multiple types of operations **in** parallel, including 1×1, 3×3, 5×5 convolutions and 3×3 max pooling. The outputs from all paths are concatenated depth-wise.

- **Purpose:** Enables the network to capture features at **multiple scales** effectively.
- **Advantage:** Improves representational power without dramatically increasing computation.

Conti…

Auxiliary Classifiers

To address the vanishing gradient problem during training, GoogLeNet introduces auxiliary classifiers(intermediate branches that act as smaller classifiers). These are active only during training and help regularize the network.

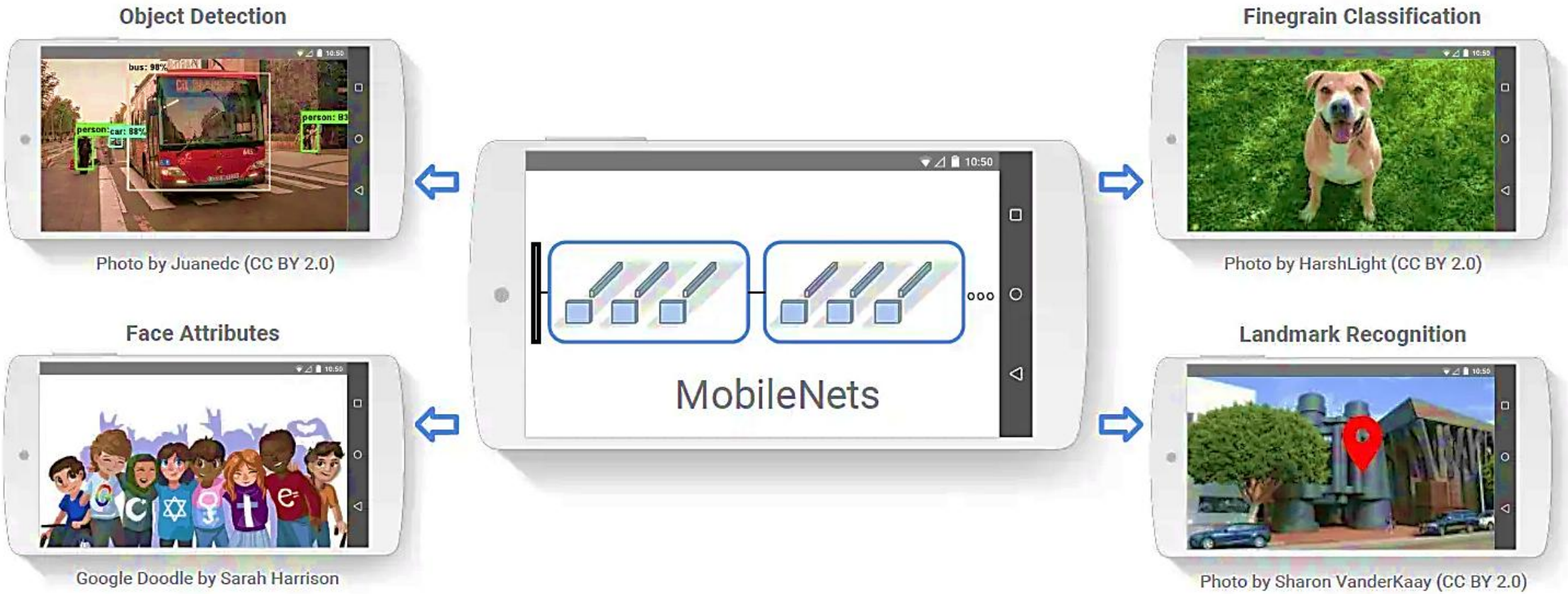
InceptionNet Architecture

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

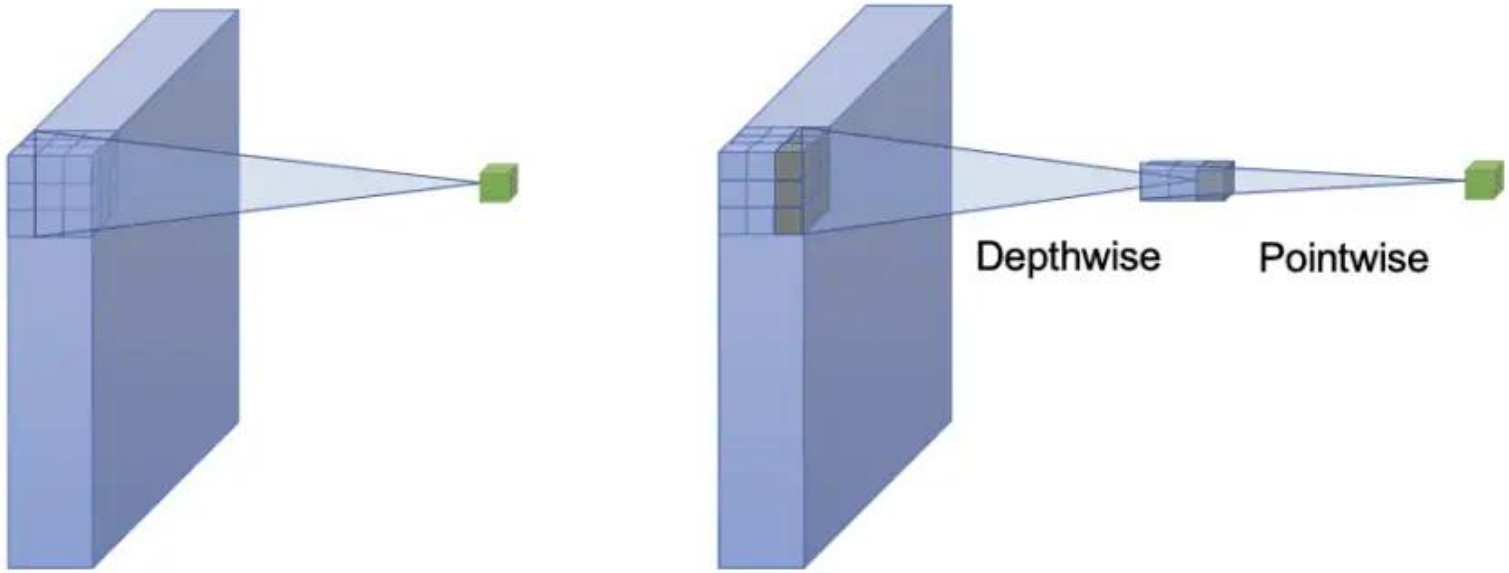
MobileNet

- **MobileNet** is a family of convolutional neural network (CNN) architectures designed for image classification, object detection, and other computer vision tasks.
- They are designed for small size, low latency, and low power consumption, making them suitable for on-device inference and edge computing on resource-constrained devices like mobile phones and embedded systems.
- They were originally designed to be run efficiently on mobile devices with TensorFlow Lite.

Conti…


Object Detection
bus: 98%
person: car: 88%
Photo by Juanedc (CC BY 2.0)
Face Attributes
Google Doodle by Sarah Harrison
10:50
MobileNets
Finegrain Classification
Photo by HarshLight (CC BY 2.0)
Landmark Recognition
Photo by Sharon VanderKaay (CC BY 2.0)

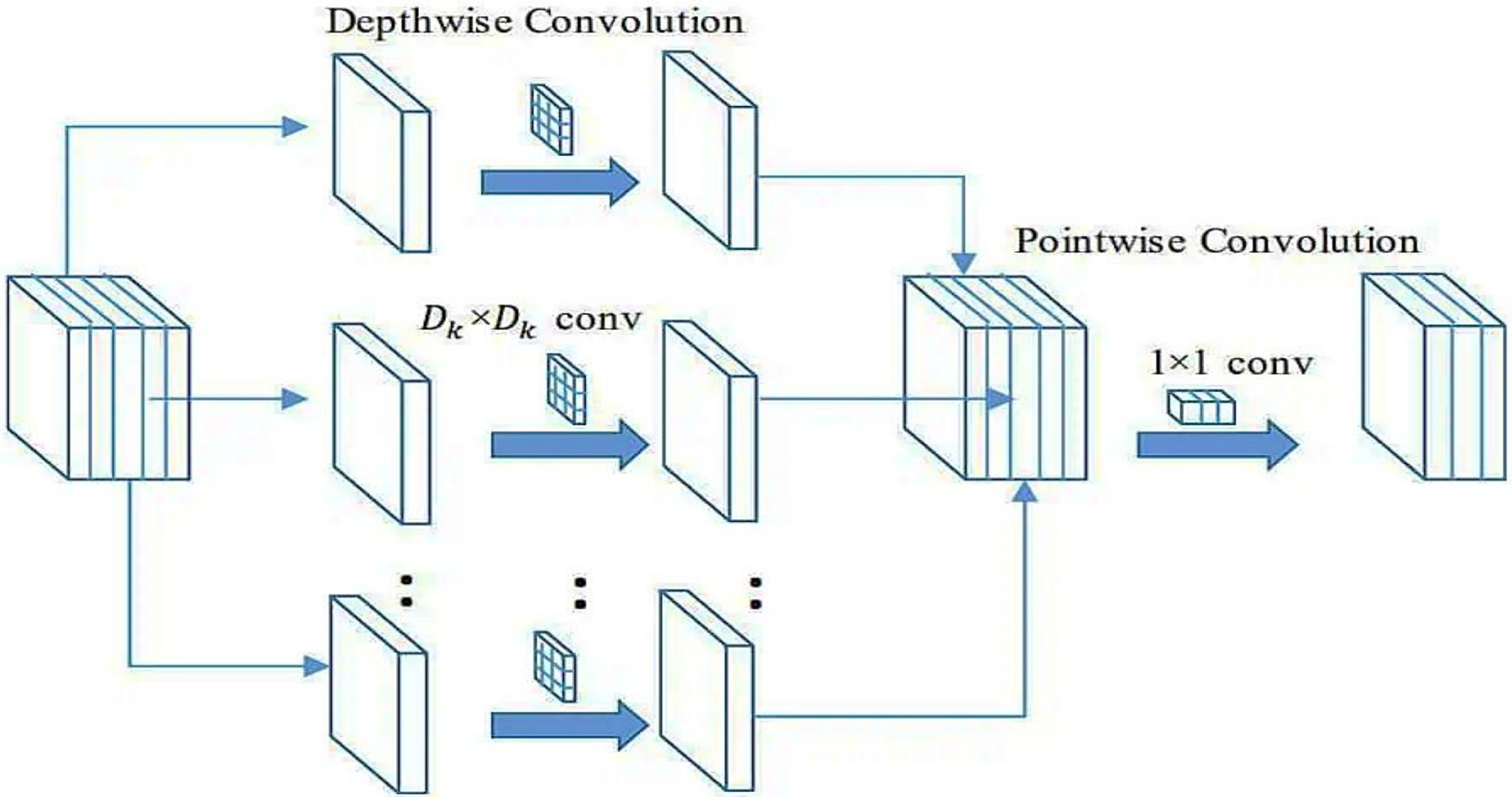
Conti…



Depthwise Separable Convolution splits the standard convolution into two separate steps:

- Depthwise convolution: Applies a single filter to each input channel.
- Pointwise convolution (1×1 convolution): Combines the outputs from the depthwise convolution to create new features.

Conti…



Depthwise Separable Convolution

Pointwise Convolution

To combine features or expand channels, the output of depthwise convolution is applied with pointwise convolution. This combines or expands the channels.

- **Combining Channel Features**: Pointwise convolution combines these independently processed channel features to create a new feature map, whereas depthwise convolution does not.
- **Increases Model Capacity**: Interactions among channels increase the network's representational capacity. The model can learn complex patterns that depend on the relationships between different feature channels.
- **Adjusting the Number of Channels**: Pointwise convolution enables increasing or decreasing the number of channels in the output feature map.

MobileNet Architecture

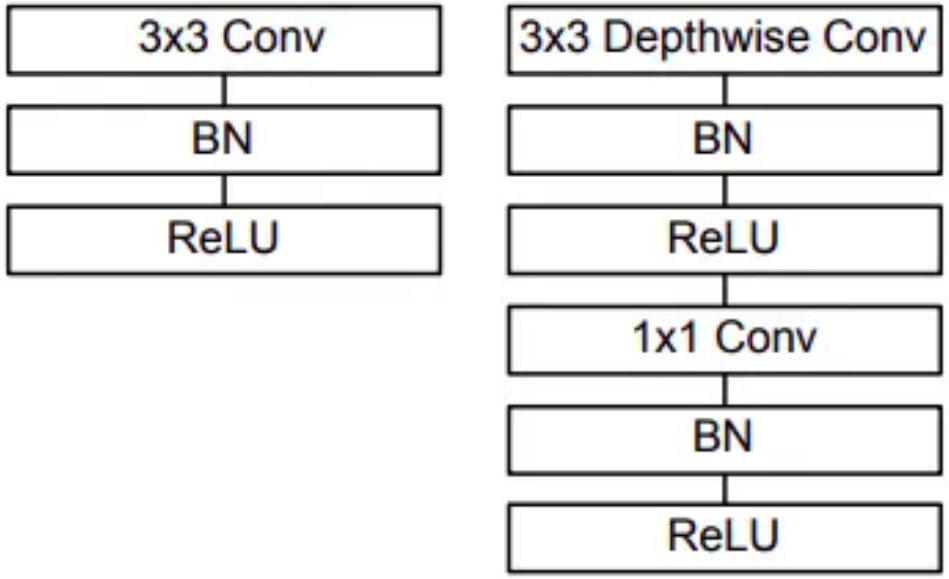
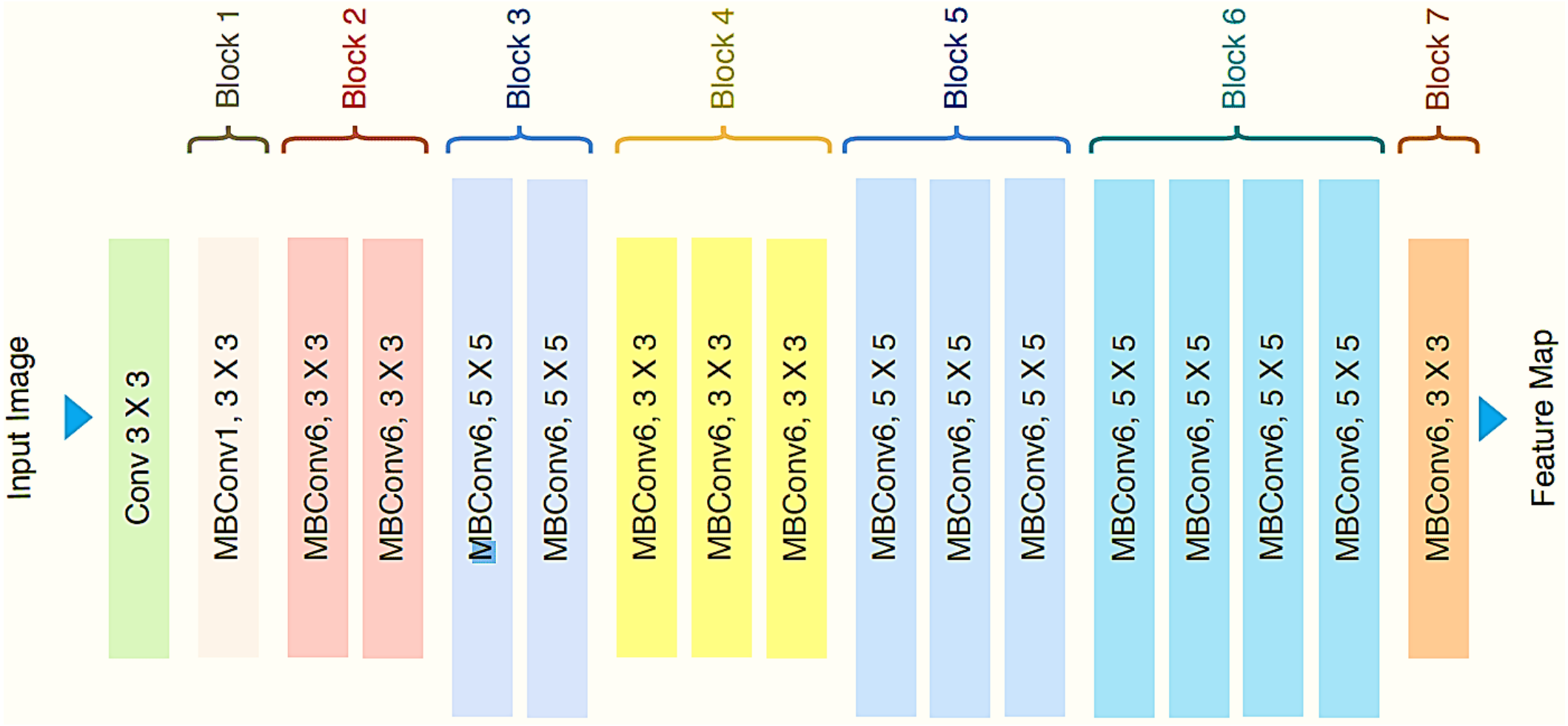


Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

EfficientNet

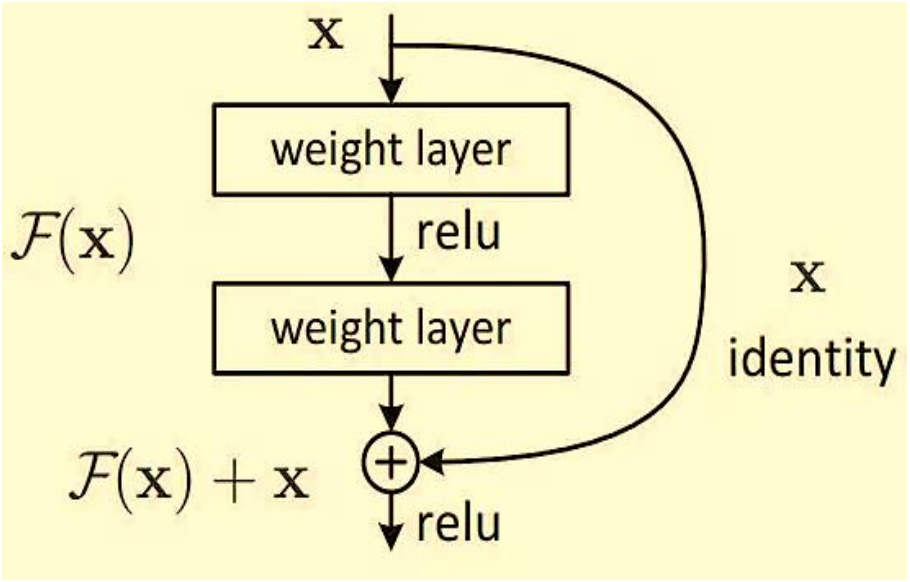


EfficientNet Architecture

The MBConv Block

The MBConv block is an evolved inverted residual block inspired by MobileNetv2.

- Residual networks (ResNets) are a type of CNN architecture that addresses the vanishing gradient problem, as the network gets deeper, the gradient diminishes. ResNets solves this problem and allows for training very deep networks.

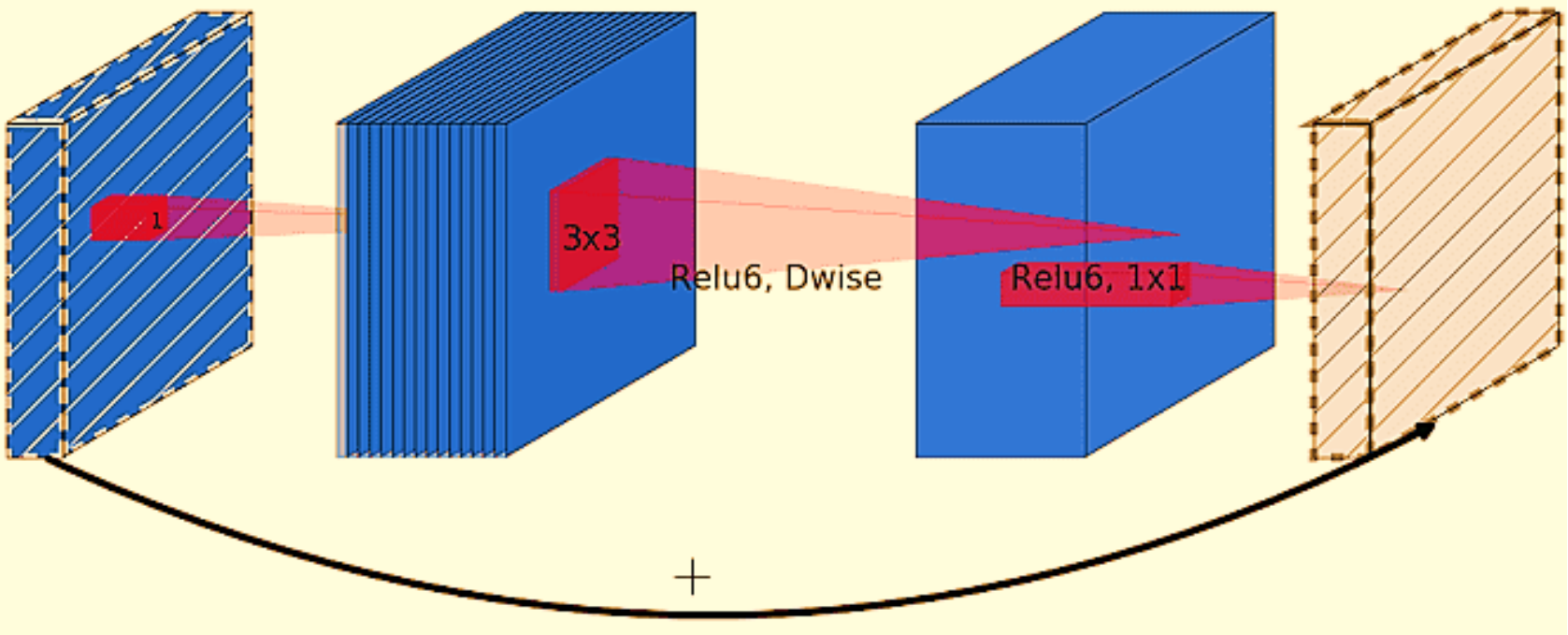


Residual Block

- This is achieved by adding the original input to the output of the transformation applied by the layer, improving gradient flow through the network.

The MBConv Block

The MBConv block is an evolved inverted residual block inspired by MobileNetv2.

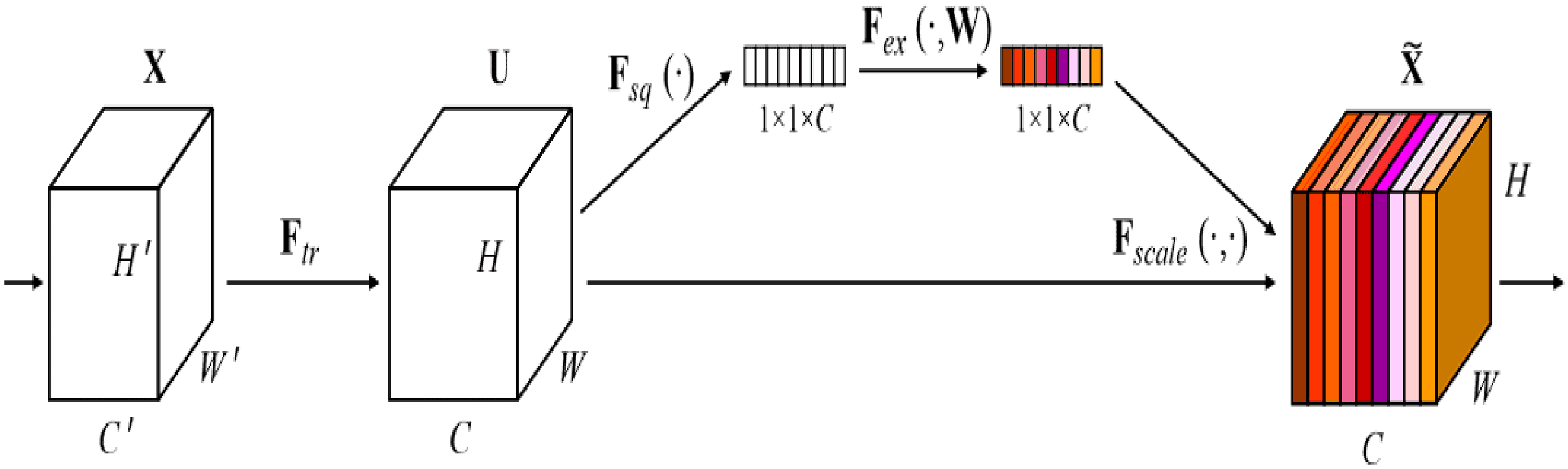


Inverted Residual Block

- This is achieved by adding the original input to the output of the transformation applied by the layer, improving gradient flow through the network.

Squeeze-and-Excitation

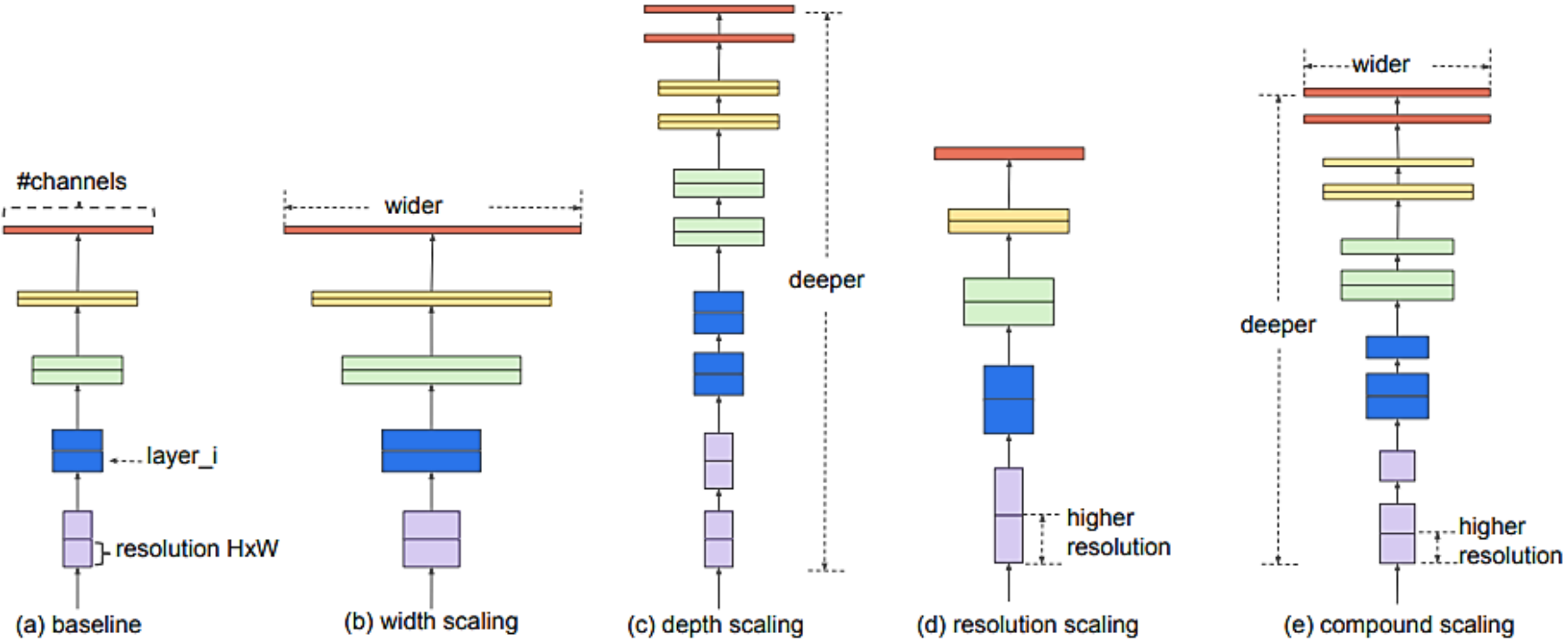
Squeeze-and-Excitation (SE) simply allows the model to emphasize useful features, and suppress the less useful ones.



Squeeze-and-Excitation

- **Squeeze:** This phase aggregates the spatial dimensions (width and height) of the feature maps across each channel into a single value, using global average pooling, reducing each channel to a single scalar value.
- **Excitation:** The model using a full-connected layer applied after the squeezing step, produces a collection of per channel weight (activations or scores), apply these learned importance scores to the original input feature map.

Squeeze-and-Excitation

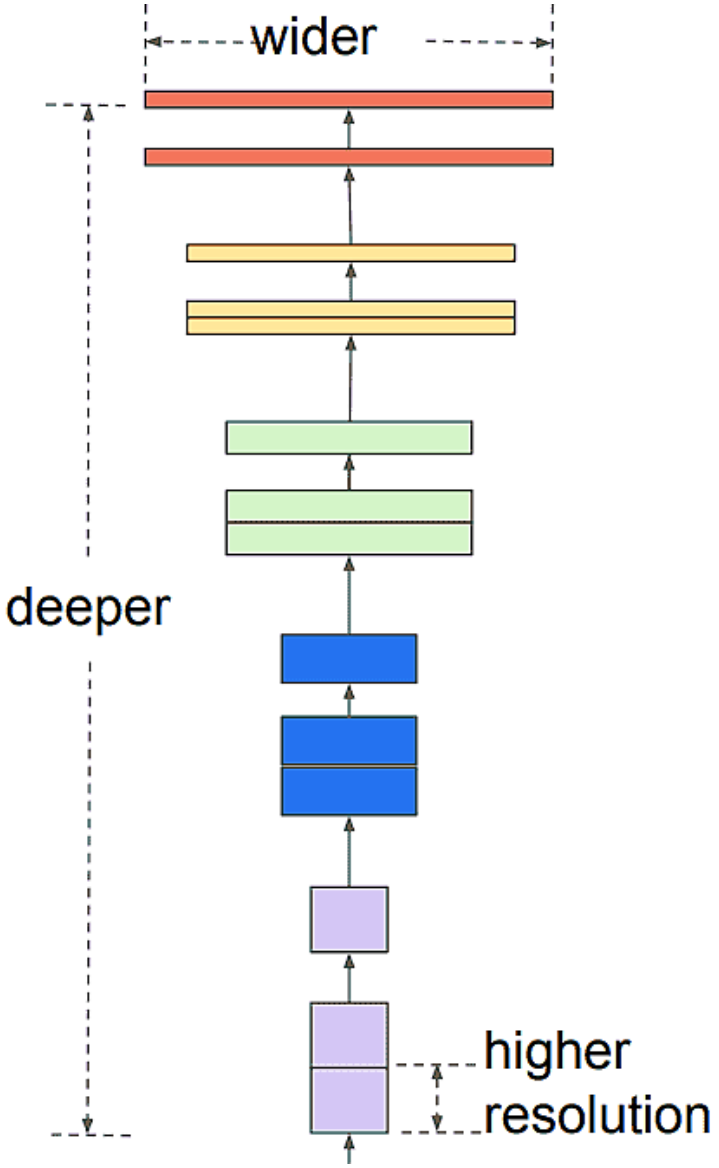


Source: https://arxiv.org/pdf/1905.11946

Compound Scaling

The creator of EfficientNet observed that different scaling dimensions (depth, width, image size) are not independent.

- High-resolution images require deeper networks to capture large-scale features with more pixels. Additionally, wider networks are needed to capture the finer details present in these high-resolution images.
- To pursue better accuracy and efficiency, it is critical to balance all dimensions of network width, depth, and resolution during ConvNet scaling.



Compound Scaling

The compound scaling coefficient method uniformly scales all three dimensions (depth, width, and resolution) in a proportional manner using a predefined compound coefficient ϕ.

α: Scaling factor for network depth (typically between 1 and 2)

β: Scaling factor for network width (typically between 1 and 2)

γ: Scaling factor for image resolution (typically between 1 and 1.5)

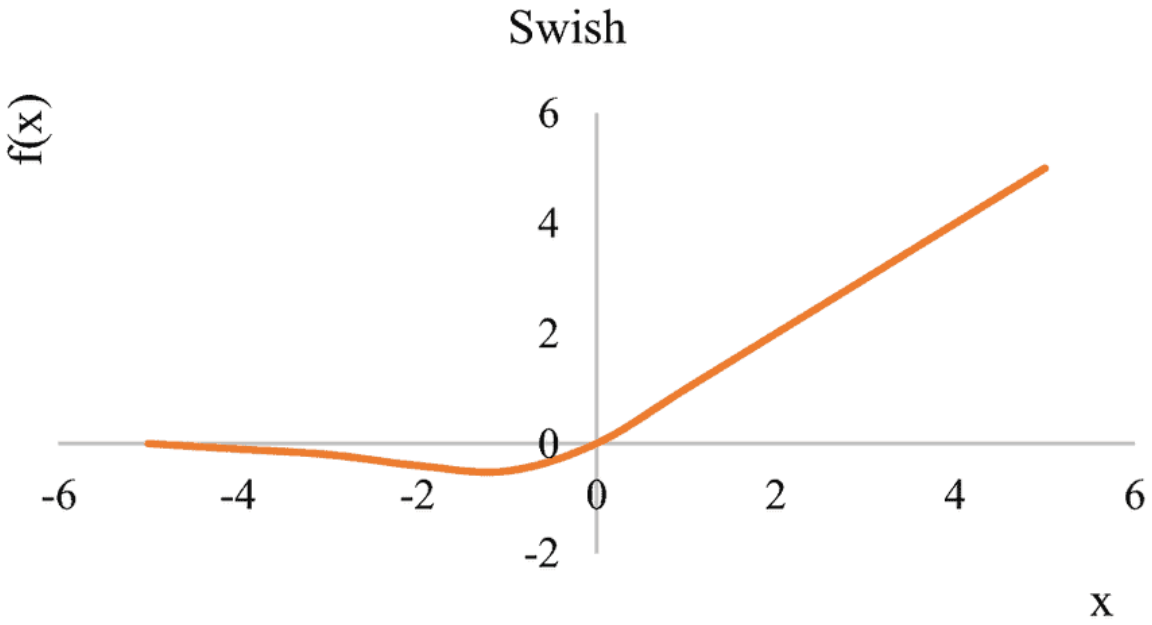
ϕ (phi): Compound coefficient (positive integer) that controls the overall scaling factor.

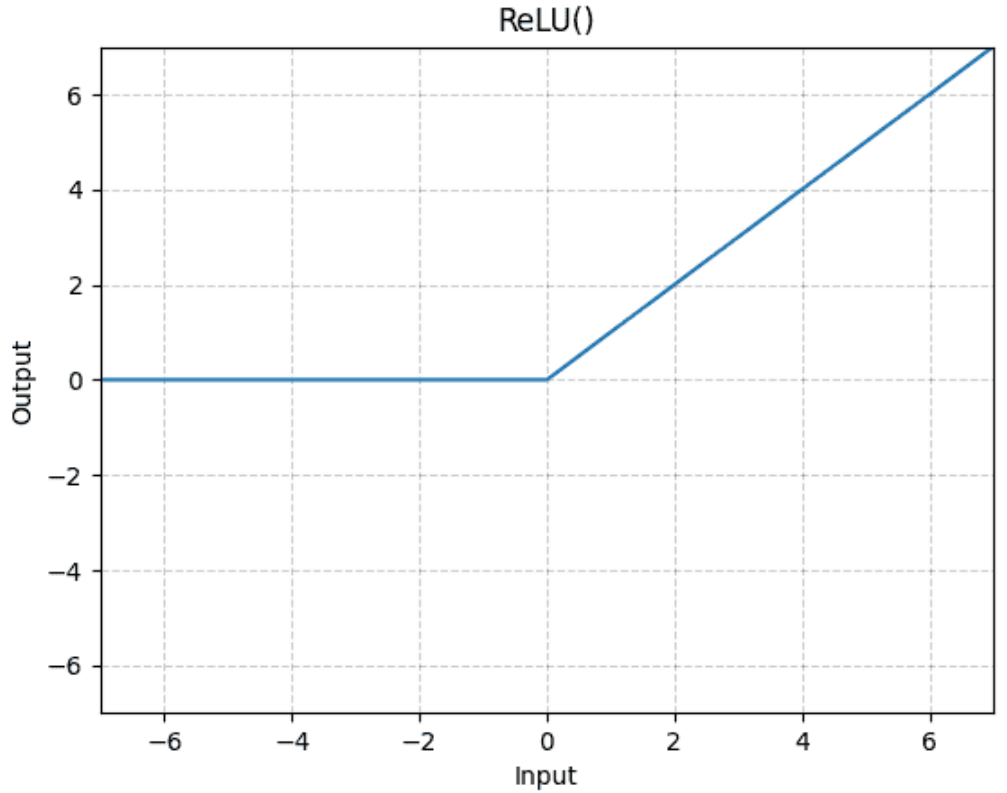
$$\begin{aligned} \text{depth: } & d = \alpha^\phi \\ \text{width: } & w = \beta^\phi \\ \text{resolution: } & r = \gamma^\phi \\ \text{s.t. } & \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \\ & \alpha \geq 1, \beta \geq 1, \gamma \geq 1 \end{aligned}$$

This equation tells us how much to scale the model (depth, width, resolution) which yields maximum performance.

Conti…

Moreover, EfficientNet also incorporates the Swish activation function as part of its design to improve accuracy and efficiency.





Conti…

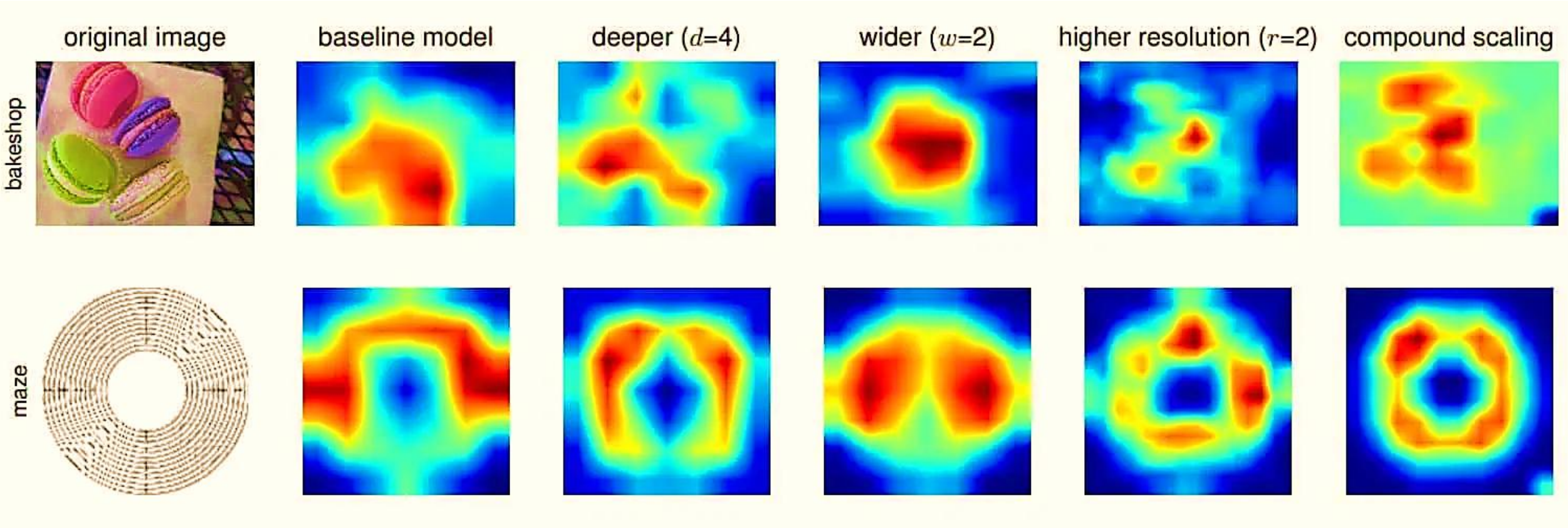
EfficientNet incorporates all the above elements into its architecture. Finally, the architecture looks like this:

Stage i	Operator $\hat{F}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224 x 224	32	1
2	MBConv1, k3x3	112 x 112	16	1
3	MBConv6, k3x3	112 x 112	24	2
4	MBConv6, k5x5	56 x 56	40	2
5	MBConv6, k3x3	28 x 28	80	3
6	MBConv6, k5x5	14 x 14	112	3
7	MBConv6, k5x5	14 x 14	192	4
8	MBConv6, k3x3	7 x 7	320	1
9	Conv1x1 & Pooling & FC	7 x 7	1280	1

EfficientNet Architecture

Conti…

Moreover, we visualize the advantage of compound scaling using an activation map.



Source: https://arxiv.org/pdf/1905.11946

Performance and Benchmarks of EfficientNet

The benchmarks are obtained on the ImageNet dataset. Here are a few key insights from the benchmark:

Higher accuracy with fewer parameters:

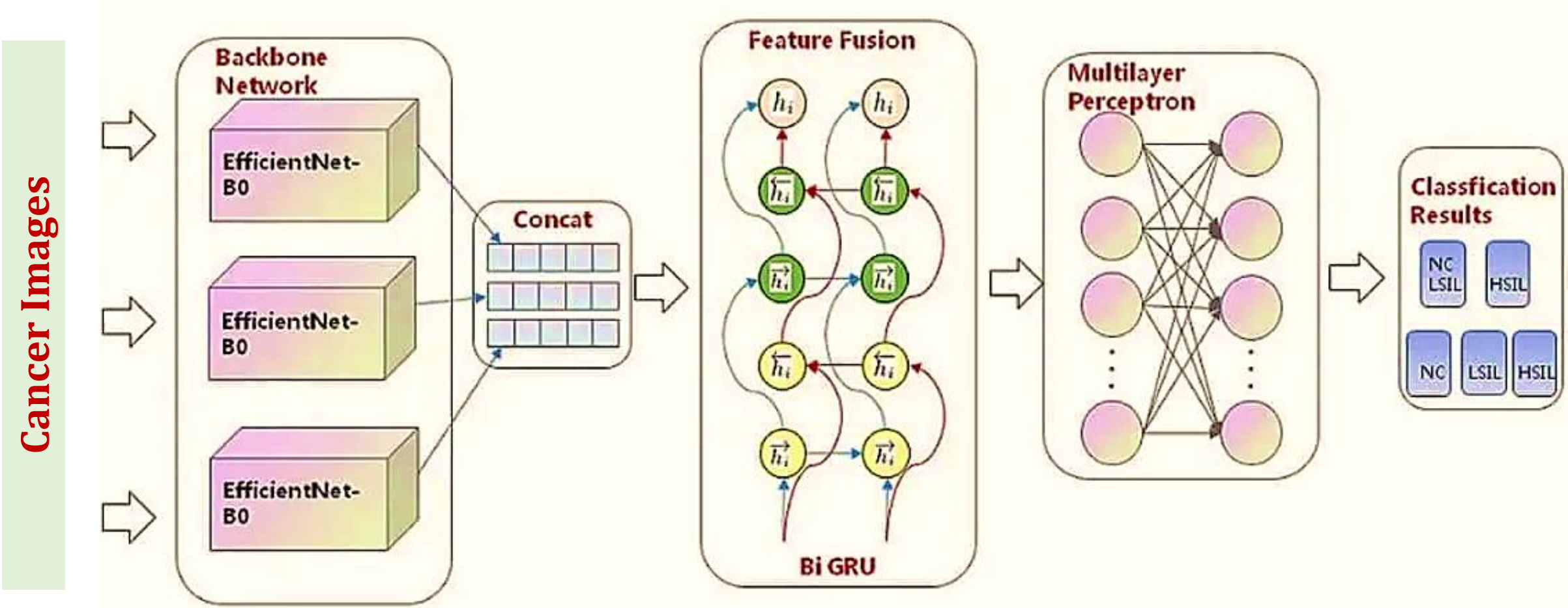
- EfficientNet models achieve high accuracy with fewer parameters and lower FLOPs than other convolutional neural networks (CNNs).
- **Ex:** EfficientNet-B0 achieves 77.1% top-1 accuracy on ImageNet with only 5.3M parameters, while ResNet-50 achieves 76.0% top-1 accuracy with 26M parameters.

Fewer Computations:

- EfficientNet models can achieve similar accuracy to other CNNs with significantly fewer FLOPs. For example, EfficientNet-B1 achieves 79.1% top-1 accuracy on ImageNet with 0.70 billion FLOPs, while Inception-v3 achieves 78.8% top-1 accuracy with 5.7 billion FLOPs.

Applications

One of the use of B0 model for cancer diagnosis, which obtained an accuracy of 91.18%



Source: 10.1002/cam4.5581